Feature Engineering

The Art of Creating Additional Features

by Saransh Saini

What is Feature Engineering?

Starting Point: Data

Real-world data is often:

- Messy
- Incomplete
- High dimensional
- Difficult for algorithms to interpret

Simply feeding raw data into a model often leads to **poor predictions.**
Good machine learning systems rely heavily on **how the data is represented.**

Do machines understand raw data the same way humans do?

Ex- “This game is absolutely terrible!!!”

A human instantly understands the **tone and meaning** but a machine only sees **text characters**.

This means we must transform raw data into **meaningful numerical representations**.

Similarly,

A dataset having **50000 columns**, can be difficult to train on. It would consume **large amount of time** and **computational power**.

Feature Engineering

A process to create new features from the existing ones.

Feature Creation

Involves building entirely new variables from raw data, like calculating **tenure** from dates or combining features to capture complex relationships. It relies on **domain knowledge** to provide the model with "hints" it wouldn't find on its own.

Feature Transformation

Focuses on resizing or reformatting data, such as **scaling** numbers to a standard range or **encoding** text labels into digits. This ensures that the mathematical algorithms can process the data efficiently without being biased by different units.

Feature Extraction

Reduces the complexity of data by condensing many variables into a smaller set of high-impact components through techniques like **PCA**. It is essential for handling **unstructured data**, such as turning raw images or text into meaningful numerical summaries.

Feature Selection

It's the process of filtering out redundant or irrelevant variables to keep only the most **predictive data**. This prevents **overfitting** and speeds up the model by focusing on the "signals" rather than the "noise."

Feature Creation

Date Decomposition: Extracting specific parts like **Is_Weekend** or **Month** from a raw timestamp to capture seasonal or weekly patterns.

Ratios: Dividing two numbers, like **Debt-to-Income**, to provide a better measure of risk than either raw value alone.

Aggregation: Calculating the **Average Spend per Transaction** by dividing total revenue by the number of orders a user placed.

Interaction Terms: Multiplying two variables, such as **Width × Height** to create **Area**, which helps the model understand their combined impact.

Recency Calculation: Subtracting the "Last Purchase Date" from today to create a **Days_Since_Active** feature to predict customer churn.

Feature Transformation

Standardization (Z-score): Centering a feature like **Income** so the mean is 0 and the standard deviation is 1, ensuring it doesn't "outweigh" smaller-scale variables.

One-Hot Encoding: Converting a categorical column like **Color** (Red, Blue) into separate binary columns **(Is_Red: 1, Is_Blue: 0)** so the model can process them.

Log Transformation: Applying a logarithm to skewed data, like **House Prices**, to compress extreme outliers and create a more normal distribution.

Min-Max Scaling: Squishing values like **Age** into a fixed range between 0 and 1, which is often required for neural networks and distance-based models.

Binning: Turning a continuous variable like **Temperature** into discrete categories (e.g., Cold, Warm, Hot) to simplify complex numerical patterns.

Feature Extraction

Principal Component Analysis (PCA): Compressing 50 correlated columns into 5 "principal components" that retain the most important data patterns.

Linear Discriminant Analysis (LDA): Reducing data into fewer dimensions by finding directions that best separate different classes.

Word Embeddings: Converting raw text like "King" and "Queen" into high-dimensional numerical vectors that represent their meaning.

TF-IDF (Term Frequency-Inverse Document Frequency): Turning a document into a score that highlights which words are uniquely important to that specific text.

Edge Detection: Using filters on raw image pixels to extract outlines and shapes, making it easier for a model to recognize objects.

Spectrogram Analysis: Converting raw audio waves into a visual frequency map to extract patterns in pitch and rhythm over time.

Feature Selection

Correlation Matrix: Dropping features that are highly correlated with each other to remove **redundant information** and simplify the model.

Lasso Regularization (L1): Using a model that automatically shrinks the coefficients of less important features to **zero**, effectively removing them.

Recursive Feature Elimination (RFE): Repeatedly training a model and removing the **weakest feature** in each round until only the most predictive ones remain.

Chi-Square Test: A statistical test used for categorical data to pick the features that have the strongest **independence relationship** with the target variable.

Tree-Based Importance: Using algorithms like **Random Forest** to calculate a score for each feature based on how much it improves the model's accuracy.

Let’s look at some of these techniques

Scaling Techniques

Standardization (Z-Score): It transforms data so that it follows a **standard distribution.** That means the mean becomes 0 and the standard deviation becomes 1.

This means:

- Values above the average become **positive numbers**
- Values below the average become **negative numbers**
- The scale becomes **consistent across features**

Ex - Income values like 30000, 45000, 90000

After standardization they become values such as -0.7, 0.1, 1.4

Min-Max Scaling: It rescales data into a **fixed range**, usually between **0 and 1**. So, the smallest value becomes 0 and the largest value becomes 1 and all other values fall somewhere between them.

This helps in keeping all features within a consistent range.

Ex- Age values 20, 30, 50

After Min-Max Scaling: 0.0, 0.33, 1.0

Log Transformation

Log Transformation is used when a feature has a **highly skewed distribution**.

In many real-world datasets, values are **not evenly distributed**.
A small number of observations may have **extremely large values (outliers)**, which creates a **long tail** in the data.

Log transformation reduces this imbalance by **compressing large values while keeping smaller values relatively spread out**.

Ex - Original values: 100 → 1000 → 10000

After applying log transformation: 2 → 3 → 4

Instead of increasing exponentially, the numbers now grow **more gradually**.

One-Hot Encoding

This converts categories into **binary indicator columns** by creating new columns for each unique category. It represents each category as a vector with a single "hot" (1) value at its index, ensuring models don't misinterpret nominal data as ordered.

Ex - Original feature: Color – Red, Blue, Green

After encoding: Is_Red, Is_Blue, Is_Green

Each row will contain: 1, if the category is present and 0 otherwise

So for a value in the original dataset becomes:

Red → (1,0,0)
Blue → (0,1,0)

Can be implemented using **pandas.get_dummies()** but we will prefer **sklearn.preprocessing.OneHotEncoder.**

PCA (Principal Component Analysis)

PCA is a **dimensionality reduction technique** that transforms a dataset into a smaller set of new variables called **principal components**.

These components:

- Capture the **maximum variance** in the data
- Are **uncorrelated with each other**
- Reduce the number of dimensions

Benefits –

- **Dimensionality Reduction:** It reduces the number of features (variables) in a dataset, which helps combat the "curse of dimensionality".
- **Faster Computation & Training:** By reducing the number of variables, machine learning algorithms run faster and require less memory.
- **Eliminates Multicollinearity:** PCA transforms data into orthogonal (uncorrelated) components, removing multicollinearity, which can hinder the performance of some models.

LDA (Linear Discriminant Analysis)

LDA is also is a **dimensionality reduction** technique used for classification problems but unlike PCA, which focuses on **variance**, LDA focuses on **class separation**.

LDA works by finding a linear combination of features that best separates two or more classes.

So, LDA tries to find directions where:

- Data points from the **same class are close together**
- Data points from **different classes are far apart**

Limitations:

- **Linearity:** As a linear method, it may struggle with non-linearly separable data, requiring alternatives like Kernelized LDA.
- **Dimensionality Constraint:** The number of linear discriminants is at most C - 1, where C is the number of classes.

Some applications include:

- Classifying emails into **Spam vs Not Spam.**
- Facial Recognition, etc.

Term Frequency - Inverse Document Frequency (TF-IDF)

TF-IDF is a numerical statistic to rank the importance of words in a document relative to a larger collection (corpus). It calculates a weight by multiplying how often a word appears in a specific document (TF) by the inverse frequency of that word across all documents (IDF).

- **TF (Term Frequency):** How often a word appears in a single comment.
- **IDF (Inverse Document Frequency):** How rare that word is across the entire dataset.
- **Result:** TF-IDF heavily penalizes common filler words and boosts rare, highly contextual words that actually define the category of the text.

Word Embeddings

TF-IDF is a numerical statistic to rank the importance of words in a document relative to a larger collection (corpus). It calculates a weight by multiplying how often a word appears in a specific document (TF) by the inverse frequency of that word across all documents (IDF).

- **The Limitation of TF-IDF:** While TF-IDF is great at finding rare words, it treats words as isolated islands. But it doesn't know that "**happy**" and "**joyful**" mean almost the same thing, or that "bad" and "awful" are related.
- **The Upgrade (Word Embeddings):** Techniques like Word2Vec or GloVe translate words into dense, multi-dimensional coordinate spaces (vectors).
- **How it Works:** The model reads millions of sentences and learns that words appearing in similar contexts have similar meanings. It physically maps them closer together in mathematical space.

Selection Techniques

Lasso Regularization (L1): It is a **regularization technique used with linear models**.

It adds a penalty that **pushes less important feature coefficients toward zero**.

Technique:

- Important features keep **non-zero weights**
- Unimportant features become **exactly zero**

So, effectively the model automatically removes irrelevant features.

Recursive Feature Elimination (RFE): Recursive Feature Elimination works by recursively removing the least important features from a dataset to build a more efficient and effective predictive model.

Steps:

- Train a model using all features
- Rank the importance of each feature
- Remove the **least important feature**
- Train the model again

This process repeats until only the **most predictive features remain**.

Selection Techniques

Chi-Square Test: It is commonly used with **categorical features** to measure how strongly a feature is **related to the target variable**.

The basic idea is that if a feature and the target are **independent**, the feature is not useful for prediction. If they are **dependent**, the feature likely contains useful information.

$$\chi^2 = \sum \left(\frac{(O - E)^2}{E} \right)$$

where:

O = Observed frequency

E = Expected frequency (if variables were independent)

Tree-Based Feature Importance: Tree-based algorithms such as **Decision Trees and Random Forests** can estimate feature importance automatically.

How it works:

- Each time a feature is used to **split the data**, the model measures how much it improves prediction accuracy.
- Features that improve the model the most receive **higher importance scores**.

Example:

In predicting house prices, important features may be:

- Location
- Size of house
- Number of bedrooms

Less useful features receive **low importance values**.

Thank you